3D WEBSOFT INC. FOUNDATION

SCHOOL FUTURE TECH PROGRAM

Faculty Name :

Course Duration : 5 Months (150 Days)

Classes : Offline & Online

Daily Teaching : 3 Hours

Total Hours : 300 Hours

LEVEL 3 –Future Engineers(9th–10th Standard)

MODULE 1 – Python Foundations (Classes 1–10)

Class 6 – Variables & Constants

Complete Theory & Practical Notes

Learning Objectives

By the end of this session, the class aims to:

1. Introduce students to the concept of variables in programming.
2. Explain why variables are needed to store and work with information.
3. Help students understand how values are assigned to variables in Python.
4. Introduce different types of values that can be stored in variables.
5. Explain the basic rules for naming Python variables.
6. Introduce the concept of constants and their purpose.
7. Differentiate between variables and constants.
8. Develop students' ability to create, update, and use variables in Python programs.
9. Provide hands-on practice using variables with text and numbers.
10. Develop basic problem-solving skills through simple variable-based programs.

Session

Topic	*Method*
Warm-up & Introduction	Discussion
What is a Variable?	Theory
Creating & Assigning Variables	Theory + Demonstration
Variable Naming Rules	Theory + Activity
My Information Program	Hands-on Practical 1
Break / Variable Challenge	Activity
Updating & Using Variables	Theory + Practical
Introduction to Constants	Theory + Demonstration
Variables & Constants	Hands-on Practical 2
Quiz + Exit Activity	Assessment

1. Warm-up & Introduction

Begin with questions students can relate to.

“If you wanted the computer to remember your name, where would you store it?”

“What if your age changes next year?”

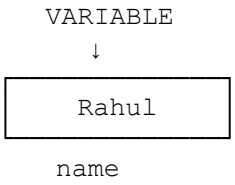
“How can a program remember a student's marks?”

Introduce the idea:

A variable is a named place used by a program to store a value.

Real-Life Analogy

Use a **labelled box** analogy.



The label is:

`name`

The stored value is:

`"Rahul"`

Explain:

A variable is like a labelled container that allows a program to store and access information.

2. What is a Variable?

THEORY

Definition

A **variable** is a name that refers to a value stored or managed by a program.

Example:

```
name = "Rahul"
```

Here:

- `name` → variable
- `"Rahul"` → value
- `=` → assignment operator

Explain that Python associates the name `name` with the value `"Rahul"`.

Another Example

```
age = 15
```

Here:

```
Variable → age
Value    → 15
```

More Examples

```
student_name = "Anita"
age = 14
height = 155.5
school = "Future School"
```

3. Creating and Assigning Variables

THEORY + DEMONSTRATION

Assignment

The = symbol is used to assign a value to a variable.

```
name = "Arjun"
```

Read this as:

“Assign the value Arjun to the variable name.”

Text Value

```
name = "Arjun"
```

Integer Value

```
age = 15
```

Decimal Value

```
height = 160.5
```

Boolean Value

```
is_student = True
```

Displaying Variables

Use `print()`:

```
name = "Arjun"
print(name)
```

Output:

```
Arjun
```

Another example:

```
age = 15
print(age)
```

Output:

```
15
```

Combining Variables

```
name = "Arjun"
age = 15

print(name)
print(age)
```

Output:

```
Arjun
15
```

4. Variable Naming Rules

THEORY + ACTIVITY

Explain that variables should follow Python's naming rules.

Rule 1 – Use Meaningful Names

Good:

```
student_name = "Rahul"
student_age = 15
```

Less meaningful:

```
x = "Rahul"
```

```
a = 15
```

For beginners, encourage names that clearly describe the information.

Rule 2 – Names Cannot Start With a Number

Incorrect:

```
1name = "Rahul"
```

Correct:

```
name1 = "Rahul"
```

Rule 3 – No Spaces

Incorrect:

```
student name = "Rahul"
```

Correct:

```
student_name = "Rahul"
```

Use an underscore _ when separating words.

Rule 4 – Avoid Special Characters

Use:

```
student_name = "Rahul"
```

Avoid names containing characters such as:

```
student-name
student@name
student$name
```

Rule 5 – Python is Case-Sensitive

These are different names:

```
name = "Rahul"
Name = "Anita"
```

`name` and `Name` are treated as different identifiers.

Rule 6 – Don't Use Python Keywords

Students should not use keywords such as:

```
if
for
while
class
return
```

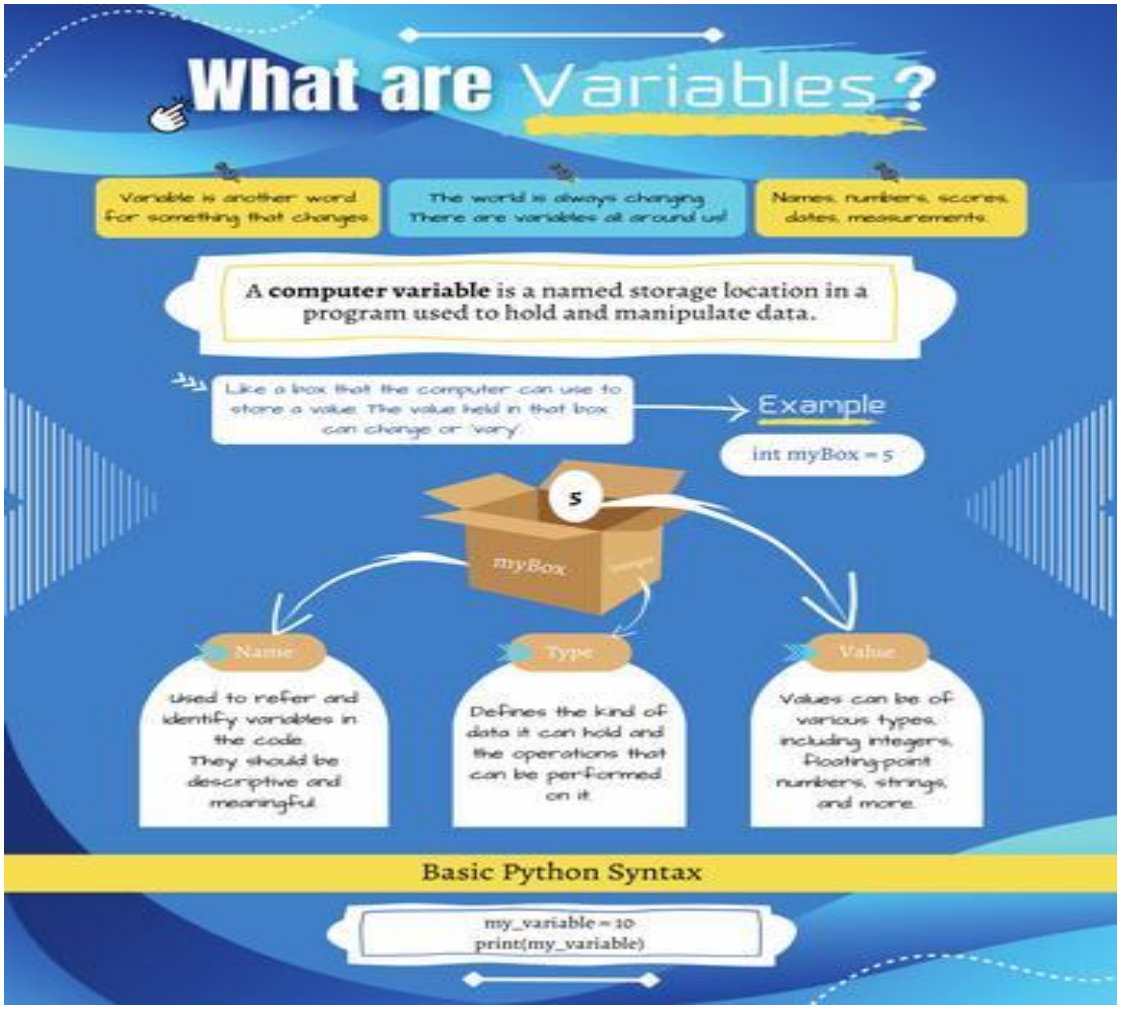
as ordinary variable names.

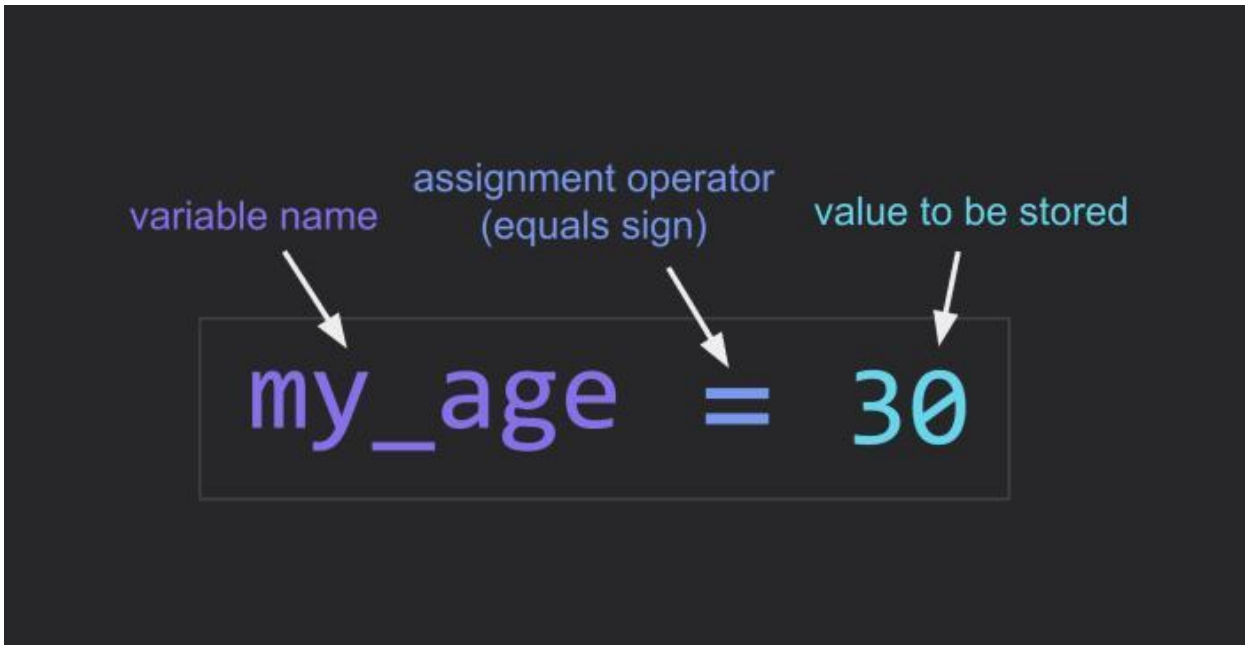
Variable Naming Activity

Ask students whether each name is **Good or Not Good**.

Variable	Good?
`name`	✓
`student_name`	✓
`age`	✓
`2name`	×
`student name`	×
`my_age`	✓
`StudentName`	✓
`student@name`	×

Suitable Visuals – Variables







Recommended classroom visual

Use a visual showing:

```
name = "Rahul"
  |          |
  |          └── VALUE
  |
  └── VARIABLE
```

Another useful visual:

```
        VARIABLE
            ↓
      ┌──────────┐
      │  Rahul   │
      └──────────┘
            ↑
          VALUE
```

This makes the concept much easier for beginners.

5. Practical 1 – My Information Program

HANDS-ON

Students create:

`my_information.py`

Starter Program

```
name = "Rahul"
age = 14
school = "Future School"
hobby = "Drawing"

print(name)
print(age)
print(school)
print(hobby)
```

Expected Output

```
Rahul
14
Future School
Drawing
```

Student Task

Students replace the values with their own information.

For example:

```
name = "__________"
age = 14
school = "__________"
hobby = "__________"

print(name)
print(age)
print(school)
print(hobby)
```

Extension

Add:

```
favorite_subject = "Science"
favorite_color = "Blue"
dream_job = "Engineer"
```

Then print them.

6. Break / Variable Challenge

Give students this code:

```
name = "Ravi"
age = 15

print(name)
print(age)
```

Ask:

“What will happen if we change name?”

Students change:

```
name = "Ravi"
```

to:

```
name = "Priya"
```

Run the program again.

Expected output:

```
Priya
15
```

Explain:

The program uses the current value associated with the variable.

7. Updating and Using Variables

THEORY + PRACTICAL

A variable's value can be changed.

Example:

```
score = 50
print(score)
```

```
score = 80
print(score)
```

Output:

```
50
80
```

Explain:

The value associated with `score` was updated from 50 to 80.

Variables in Calculations

Variables can store numbers.

```
a = 10
b = 20

total = a + b

print(total)
```

Output:

```
30
```

Another Example

```
price = 100
quantity = 3

total = price * quantity

print(total)
```

Output:

```
300
```

This demonstrates that variables are useful for **calculations**, not just storing names.

8. Practical Mini Challenge – Student Marks

marks.py

Code:

```
maths = 85
science = 90
english = 80

total = maths + science + english

print(total)
```

Expected output:

```
255
```

Extension Challenge

Calculate the average:

```
maths = 85
science = 90
english = 80

total = maths + science + english
average = total / 3

print(total)
print(average)
```

This gives students an early connection between variables and mathematical operations.

9. Introduction to Constants

THEORY

What is a Constant?

A constant is a value that is intended to remain unchanged throughout a program.

Python does not have a special constant keyword that prevents reassignment. Instead, programmers follow a naming convention: constants are usually written in uppercase letters.

Example:

```
PI = 3.14159
```

Another example:

```
MAX_SCORE = 100
```

Another:

```
SCHOOL_NAME = "Future School"
```

Explain:

```
Variable:
age = 15

Constant:
MAX_SCORE = 100
```

Important Difference

Variable	Constant
Value may change	Intended to remain unchanged
Usually lowercase/snake_case	Usually UPPERCASE
Example: `age`	Example: `MAX_SCORE`

10. Variables and Constants

HANDS-ON Practical 2

Students create:

```
variables_constants.py
```

Activity

```
student_name = "Rahul"
age = 15

MAX_SCORE = 100
SCHOOL_NAME = "Future School"

print(student_name)
print(age)
print(MAX_SCORE)
```

```
print(SCHOOL_NAME
```

Challenge

1. Which are variables?
2. Which are constants?
3. Which values are text?
4. Which values are numbers?

Expected:

```
Variables:
student_name
age

Constants:
MAX_SCORE
SCHOOL_NAME
```

11. Mini Project – Student Profile

For the final practical task, students create a small profile program.

File

`student_profile.py`

Example

```
STUDENT_SCHOOL = "Future School"
MAX_MARKS = 100

name = "Rahul"
age = 15
maths_marks = 85

print("===== STUDENT PROFILE =====")
print("Name:", name)
print("Age:", age)
print("School:", STUDENT_SCHOOL)
print("Maths Marks:", maths_marks)
print("Maximum Marks:", MAX_MARKS)
```

Expected Output

```
===== STUDENT PROFILE =====
Name: Rahul
Age: 15
School: Future School
Maths Marks: 85
Maximum Marks: 100
```

This provides a useful transition toward the next Python topics.

12 . Quick Assessment

Q1. What is a variable?

A name associated with a value that a program can use.

Q2. What does = do?

It assigns a value to a variable.

Q3. Which is a valid variable name?

A. `2name`
B. `student name`
C. `student_name`

Answer: C

Q4. What is a constant?

A value intended to remain unchanged during a program.

Q5. How are constants commonly written in Python?

Using uppercase names, such as:

```
MAX_SCORE = 100
```

Q6. What is the output?

```
age = 15
print(age)
```

Answer:

```
15
```

13. Classroom Activity – "Variable Detective"

Give students this program:

```
name = "Anita"
age = 14
city = "Hyderabad"

MAX_AGE = 100
```

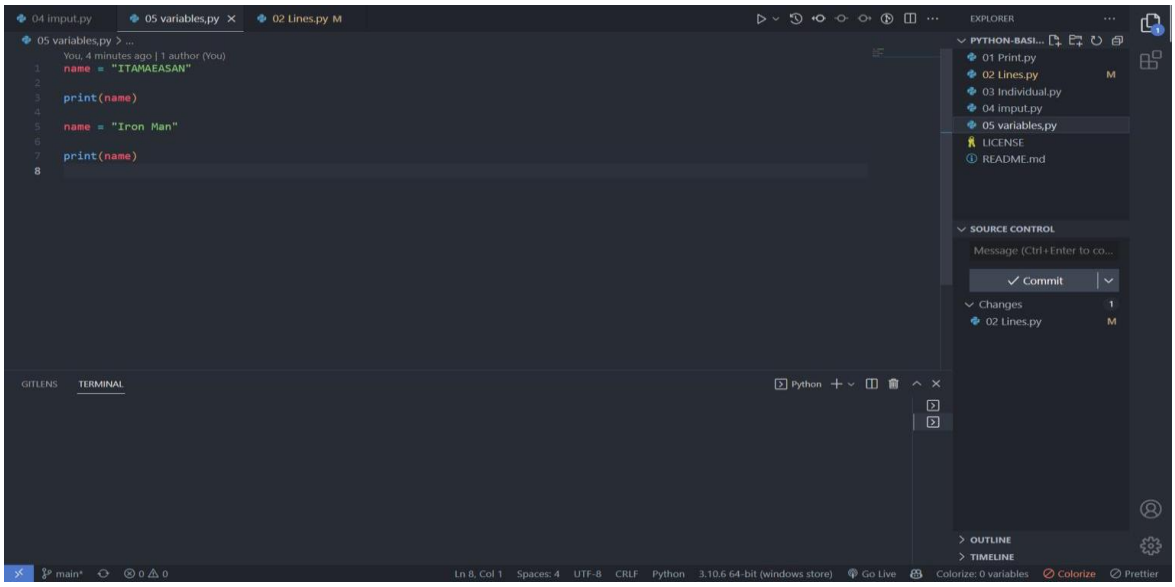
```
print(name)
print(age)
print(city)
print(MAX_AGE)
```

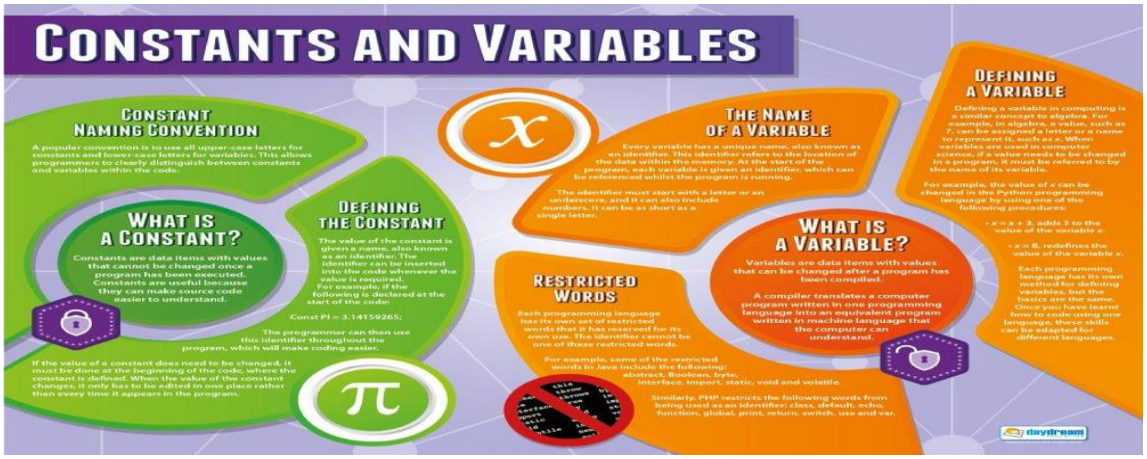
Students identify:

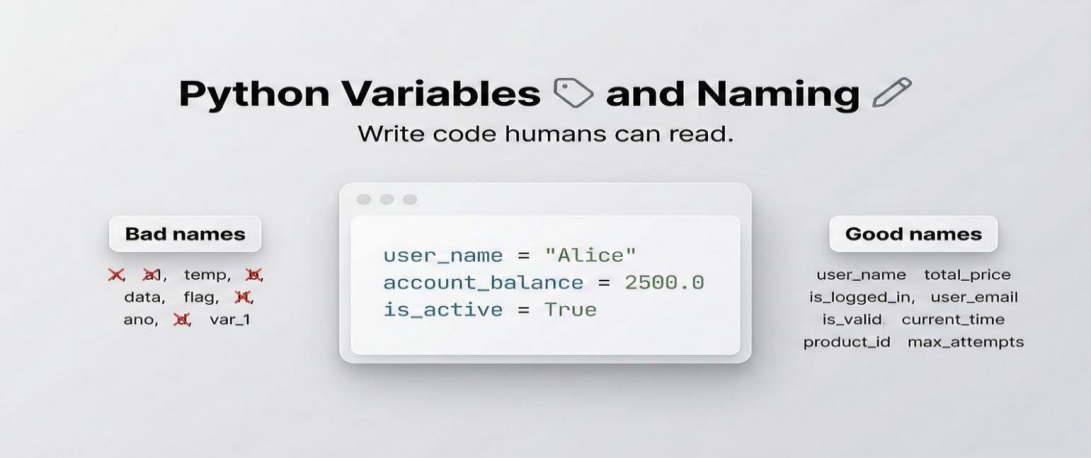
- Variables
- Constants
- Text values
- Number values
- Output

This can be done as a pair activity.

14. Suggested Images for PPT







15. Teacher Preparation

Before the class, ensure:

- Python is installed.
- VS Code is installed.
- Python extension is enabled.
- Students can create and run .py files.
- A sample project folder is prepared.
- PPT/visuals are ready.
- Practical exercises are distributed.

Suggested Folder

```
Python_Foundations
└── Class01
    ├── my_information.py
    ├── marks.py
    ├── variables_constants.py
    └── student_profile.py
```

Learning Outcomes

By the end of this class, students will be able to:

1. **Explain** what a variable is in Python.
2. **Describe** how variables are used to store and work with information.
3. **Create** variables using Python assignment statements.
4. **Assign** text, integer, decimal, and Boolean values to variables.

5. **Display** variable values using print().
6. **Change and update** the value of a variable.
7. **Use variables** in simple mathematical calculations.
8. **Apply** basic Python variable naming rules.
9. **Identify** valid and invalid variable names.
10. **Explain** the concept of a constant.
11. **Differentiate** between variables and constants.
12. **Use uppercase naming conventions** when representing constants.
13. **Create** a simple Python program using multiple variables and constants.
14. **Run and test** a variable-based Python program in VS Code.
15. **Demonstrate** basic problem-solving skills by modifying variable values and observing the output.